

Руководство программиста

Архитектура исполнения: POLIZ + VM

12 февраля 2026 г.

Содержание

1	Архитектура проекта	3
2	Данные компилятора: таблицы символов	3
2.1	Переменные (TID)	3
2.2	Функции (TFunc)	3
3	POLIZ: модель инструкций	3
3.1	Стековая семантика	3
4	VM: состояние и память	4
4.1	Instruction Pointer (ip)	4
4.2	Стек значений	4
4.3	Память mem_	4
4.4	Кадры функций (frames_)	4
5	Ключевые функции vm.cpp	4
5.1	eval(Value v)	4
5.2	readTyped(Address a) и writeTyped(Address a, Value v)	4
5.3	Построение lvalue: ADDRESS_*	5
5.4	Присваивание	5
5.5	Управляющие переходы: POLIZ_GO и POLIZ_FGO	5
5.6	Вызовы функций	5
5.7	Почему важен “final FREE” в конце функций	6
5.8	Встроенные I/O: CALL_PRINT, CALL_READ	6
6	Типичные причины падений и способы отладки	6
6.1	Memory OOB	6
6.2	Heap corruption (Windows 0xC0000374)	6
7	Мини-спецификация соглашений	6
7.1	Модель памяти	6
7.2	Вызовы функций	6

8	Грамматика языка (по синтаксеру)	7
8.1	Лексические токены	7
8.2	EBNF	7
8.3	Замечания по соответствию реализации	8

1 Архитектура проекта

Проект реализует компиляцию и выполнение учебного языка на базе:

- **Lexer** — токенизация исходного текста;
- **Syntaxer/Parser** — разбор грамматики, семантика, построение таблиц символов;
- **POLIZ** — линейный промежуточный код (стековый байткод);
- **VM (PolizVm)** — стековая виртуальная машина исполнения POLIZ.

2 Данные компилятора: таблицы символов

2.1 Переменные (TID)

Каждая переменная хранится как элемент таблицы идентификаторов:

- тип (`int/double/bool/char`);
- смещение `offset` в памяти VM (в байтах);
- признаки массива (если массив: размер и тип элемента).

2.2 Функции (TFunc)

Для функции сохраняются:

- имя;
- тип возвращаемого значения;
- список параметров (каждый параметр имеет тип и смещение в кадре);
- адрес входа в **POLIZ** (`entry ip`) — индекс инструкции, на которую прыгает VM.

3 POLIZ: модель инструкций

POLIZ хранит программу как последовательность элементов вида:

$$(\text{op}, \text{arg})$$

где `op` — код операции (`enum/число`), `arg` — строковый аргумент (часто пустой).

3.1 Стековая семантика

Большинство операций работают со стеком:

- литералы кладут значение на стек;
- бинарные операции снимают два значения и кладут результат;
- присваивание снимает `rhs` и адрес `lhs`, пишет в память, и кладёт `rhs` обратно (для цепочек присваивания).

4 VM: состояние и память

4.1 Instruction Pointer (ip)

`ip` — индекс текущей инструкции в `POLIZ`.

4.2 Стек значений

Стек хранит значения и адреса. Типовая реализация — `std::variant`. На стеке могут лежать:

- значения (`int/double/bool/char`);
- адреса (структура `Address{abs, type}`), где `abs` — абсолютный адрес в памяти.

4.3 Память `mem_`

Память VM — `vector<uint8_t>` (байтовый буфер). Доступ выполняется через `memcpu` (или шаблоны `readPod/writePod`). Проверки выхода за границы осуществляются сравнением:

$$abs + sizeof(type) \leq |mem_|$$

4.4 Кадры функций (`frames_`)

При вызове функции VM создаёт новый кадр (`frame`), который хранит:

- `bp` — base pointer: начало памяти кадра;
- `sp` — stack pointer кадра (текущий конец выделенной памяти в байтах);
- дополнительную метаинформацию, например список `offsets` параметров-массивов.

Освобождение кадра означает `mem_.resize(bp)` и `pop_back()` кадра.

5 Ключевые функции `vm.cpp`

Ниже описаны функции и их смысл на уровне реализации.

5.1 `eval(Value v)`

Если на стеке лежит **адрес**, `eval` разыменовывает его, читая из `mem_`. Если лежит значение — возвращает его как есть. Это позволяет операциям прозрачно работать с идентификаторами.

5.2 `readTyped(Address a)` и `writeTyped(Address a, Value v)`

`readTyped` читает значение из памяти по адресу и типу. `writeTyped` записывает значение в память, приводя типы по правилам языка. Именно эти функции являются основной точкой контроля памяти и ООВ-проверок.

5.3 Построение lvalue: ADDRESS_*

Инструкции ADDRESS_INT/DOUBLE/BOOL/CHAR кладут на стек структуру Address:

$$Address\{abs, type\}, \quad abs = frame.bp + offset$$

Для массивов offset вычисляется как:

$$offset = baseOffset + index \cdot sizeof(elementType)$$

5.4 Присваивание

Присваивание реализуется в обработке операции “=”:

1. снять rhs со стека, при необходимости разыменовать;
2. снять lhs как Address;
3. `writeTyped(lhs, rhs);`
4. положить rhs обратно на стек (для цепочек `a=b=c`).

5.5 Управляющие переходы: POLIZ_GO и POLIZ_FGO

GO — безусловный переход: `ip = target`. FGO — переход по ложному условию:

1. получить условие (bool) и target (label);
2. если условие ложно — `ip = target`, иначе продолжить.

Так компилируются `if/while/for`.

5.6 Вызовы функций

CALL_FUNCTION выполняет:

1. получить адрес функции (entry ip) со стека;
2. по адресу найти описание функции в TFunc;
3. снять аргументы со стека;
4. создать кадр, выделить память;
5. записать параметры в кадр:
 - скалярные параметры — по значению;
 - массив-параметр — как указатель (адрес вызывающего кадра).
6. выполнить тело функции (вызов `exes` на диапазоне инструкций);
7. получить return value и вернуть его на стек вызывающего;
8. откатить кадр (освободить память).

5.7 Почему важен “final FREE” в конце функций

В некоторых компоновках POLIZ компилятор добавляет `FREE` в конце тела функции. При этом VM сама обязана уничтожить кадр при возврате. Если выполнить и `FREE` и откат кадра VM, возможна двойная очистка памяти и краши. Решение: финальный эпилог функции не должен разрушать кадр повторно (например, VM не исполняет последний `FREE` в теле функции, а откат кадра делает сама).

5.8 Встроенные I/O: `CALL_PRINT`, `CALL_READ`

- `CALL_PRINT`: снять значение и вывести в `stdout`.
- `CALL_READ`: снять `Address`, прочитать из `stdin`, записать в память через `writeTyped`.

6 Типичные причины падений и способы отладки

6.1 Memory OOB

Причины:

- неверные `offsets` (особенно в массивах и параметрах функций);
- неверные размеры типов (например, `double` выделено как `int`);
- ошибочный порядок снятия `label/condition` в `FGO`.

6.2 Heap corruption (Windows 0xC0000374)

Часто является следствием предыдущих проблем, когда `memscr` пишет не в тот диапазон. Рекомендуемые инструменты:

- `_GLIBCXX_DEBUG` (GCC/libstdc++);
- `AddressSanitizer` (`-fsanitize=address`) (если доступен).

7 Мини-спецификация соглашений

7.1 Модель памяти

- `offsets` хранятся в байтах;
- скаляры размещаются плотно: `bool/char` — 1 байт, `int` — 4, `double` — 8;
- массивы — `N * sizeof(element)`.

7.2 Вызовы функций

- скаляры — по значению;
- массивы — как указатель на память вызывающего кадра.

8 Грамматика языка (по синтаксеру)

Ниже приведена грамматика в форме EBNF, соответствующая реализованным конструкциям синтаксера (объявления, присваивания, массивы, if/else, while, for, функции, return, вызовы, print/read).

8.1 Лексические токены

- ID — идентификатор;
- INT_LIT — целочисленный литерал;
- DOUBLE_LIT — литерал с точкой (если включён тип double);
- CHAR_LIT — символьный литерал.

8.2 EBNF

```
Program ::= TopLevelList EOF ;
TopLevelList ::= { TopLevelItem } ;
TopLevelItem ::= FunctionDef | DeclarationStmt ;

Type ::= "int" | "double" | "bool" | "char" ;

DeclarationStmt ::= Type DeclList ";" ;
DeclList ::= DeclItem { "," DeclItem } ;
DeclItem ::= ID ArraySuffix? InitOpt ;
ArraySuffix ::= "[" INT_LIT "]" ;
InitOpt ::= | "=" Expr ;

FunctionDef ::= Type ID "(" ParamListOpt ")" Block ;
ParamListOpt ::= | ParamList ;
ParamList ::= Param { "," Param } ;
Param ::= Type ID ParamArrayOpt ;
ParamArrayOpt ::= | "[" "]" ;

Block ::= "{" StmtList "}" ;
StmtList ::= { Stmt } ;
Stmt ::= Block
      | DeclarationStmt
      | IfStmt
      | WhileStmt
      | ForStmt
      | ReturnStmt
      | ExprStmt
      | ";" ;

IfStmt ::= "if" "(" Expr ")" Stmt ElseOpt ;
ElseOpt ::= | "else" Stmt ;

WhileStmt ::= "while" "(" Expr ")" Stmt ;
```

```

ForStmt ::= "for" "(" ForInitOpt ";" ForCondOpt ";" ForStepOpt ")" Stmt ;
ForInitOpt ::= | DeclarationInFor | Expr ;
DeclarationInFor ::= Type DeclList ;
ForCondOpt ::= | Expr ;
ForStepOpt ::= | Expr ;

ReturnStmt ::= "return" ReturnExprOpt ";" ;
ReturnExprOpt ::= | Expr ;

ExprStmt ::= Expr ";" ;

Expr ::= Assignment ;
Assignment ::= LogicalOr | LValue "=" Assignment ;
LValue ::= ID | ID "[" Expr "]" ;

LogicalOr ::= LogicalAnd { "||" LogicalAnd } ;
LogicalAnd ::= Equality { "&&" Equality } ;

Equality ::= Relational { ("==" | "!=") Relational } ;
Relational ::= Additive {("<" | ">" | "<=" | ">=") Additive } ;

Additive ::= Multiplicative { ("+" | "-") Multiplicative } ;
Multiplicative ::= Unary { ("*" | "/" | "%") Unary } ;

Unary ::= ("!" | "+" | "-") Unary | Primary ;

Primary ::= INT_LIT
          | DOUBLE_LIT
          | CHAR_LIT
          | ID
          | ID "[" Expr "]"
          | Call
          | "(" Expr ")" ;

Call ::= ID "(" ArgListOpt ")" ;
ArgListOpt ::= | ArgList ;
ArgList ::= Expr { "," Expr } ;

```

8.3 Замечания по соответствию реализации

- **Размер массива** в объявлении — литерал ([INT_LIT]), а не произвольное выражение.
- `print(...)` и `read(x)` по синтаксису являются вызовами (Call), но семантически компилируются в специальные VM-инструкции (CALL_PRINT, CALL_READ).
- Переменная цикла в форме `for(i=...)` должна быть объявлена в текущей области видимости (либо использовать форму `for(int i=...)`).